

MEMORIA TÉCNICA

SignaTour

Frontend React para Itinerarios Culturales Accesibles

Autora: Olatz González

Fecha: Mayo 2026

Repositorio: github.com/olatzglez/signatour_react

Bootcamp: The Bridge — Full Stack Developer 2026

Índice

Índice.....	2
1. Introducción y Contexto.....	2
1.1 Antecedentes y Definición del Problema.....	2
1.2 Estado del Arte y Justificación.....	3
2. Análisis de Requisitos.....	4
2.1 Requisitos Funcionales (RF).....	4
2.2 Requisitos No Funcionales (RNF).....	5
3. Planificación y Gestión del Tiempo.....	6
3.1 Desglose de Tareas (Backlog).....	6
4. Análisis Funcional y Flujos.....	7
4.1 Actores del Sistema.....	7
4.2 Interfaz de Usuario (UI).....	7
5. Arquitectura y Stack Tecnológico.....	8
5.1 Arquitectura de Componentes.....	8
5.2 Stack Tecnológico y Justificación.....	9
6. Accesibilidad y Usabilidad.....	10
6.1 Cumplimiento WCAG AA.....	10
6.2 Decisiones de Usabilidad.....	10
7. Retos Técnicos y Soluciones.....	12
Reto 1: Sincronización del estado de sesión entre componentes.....	12
Reto 2: Especificidad CSS en modo oscuro con CSS Modules.....	12
Reto 3: Accesibilidad del menú hamburguesa.....	12
9. Conclusiones y Futuros Pasos.....	14
9.1 Conclusiones.....	14
9.2 Mejoras Futuras.....	14
10. Uso de la Inteligencia Artificial.....	14
10.1 Filosofía de Uso.....	14
10.2 Tareas para las que se ha usado la IA.....	14
10.3 Tareas realizadas bajo criterio propio.....	15
10.4 Aprendizaje del Uso Responsable de la IA.....	15
11. Aprendizajes Principales.....	16

1. Introducción y Contexto

1.1 Antecedentes y Definición del Problema

Las personas sordas o con discapacidad auditiva representan aproximadamente el 5% de la población mundial según la Organización Mundial de la Salud. En España, más de un millón de personas utilizan la Lengua de Signos Española (LSE) como primera lengua o como recurso de comunicación habitual.

A pesar de los avances en accesibilidad física en museos y espacios culturales, la planificación de visitas culturales accesibles sigue siendo un proceso fragmentado y difícil para este colectivo. Los problemas principales identificados son:

- Falta de información centralizada sobre recursos de accesibilidad auditiva (LSE, subtítulos, signoguías, bucle magnético, lectura fácil) en cada espacio cultural.
- Ausencia de plataformas digitales especializadas que combinen itinerarios culturales con filtros de accesibilidad específicos para personas sordas.
- Las webs de museos y espacios culturales no siempre detallan qué recursos de accesibilidad auditiva ofrecen, obligando al usuario a contactar directamente con cada institución.

Una breve investigación exploratoria reveló que las personas sordas que planifican visitas culturales suelen recurrir a grupos de WhatsApp o foros de la comunidad sorda para intercambiar información sobre qué museos o rutas tienen intérpretes de LSE o signoguías disponibles. Esta necesidad de compartir conocimiento de forma informal evidencia la ausencia de una herramienta digital que cubra este nicho.

1.2 Estado del Arte y Justificación

Antes del desarrollo, se analizaron las soluciones existentes:

- Guías de turismo accesible generales (como Turismo Accesible o Accessible Tourism Spain): ofrecen información sobre accesibilidad física (rampas, ascensores) pero apenas contemplan la accesibilidad auditiva.
- Webs de museos nacionales: incluyen secciones de accesibilidad, pero la información está dispersa y no permite comparar ni filtrar por tipo de recurso auditivo.
- Aplicaciones de planificación de viajes (TripAdvisor, Google Maps): no contemplan filtros de accesibilidad auditiva en sus búsquedas.

SignaTour nace para cubrir este hueco. Es una plataforma web que centraliza itinerarios culturales validados con recursos de accesibilidad auditiva, permitiendo a las personas sordas o con discapacidad auditiva descubrir y planificar visitas culturales de forma autónoma.

2. Análisis de Requisitos

Para garantizar la viabilidad del MVP en el plazo de una semana, se establecieron los siguientes requisitos formales.

2.1 Requisitos Funcionales (RF)

ID	Requisito	Descripción
RF-01	Registro de usuario	El sistema permite crear una cuenta con nombre, email y contraseña con validación de formato y seguridad.
RF-02	Inicio de sesión	El sistema autentica al usuario mediante email y contraseña, persistiendo la sesión en localStorage.
RF-03	Cierre de sesión	El usuario puede cerrar sesión eliminando los datos del localStorage.
RF-04	Listado de itinerarios	El sistema muestra todos los itinerarios disponibles en un grid responsive con imagen, título y descripción.
RF-05	Detalle de itinerario	El usuario puede acceder al detalle de un itinerario con información completa: ciudad, duración y público objetivo.
RF-06	Navegación por ciudades	La home presenta un slider de ciudades disponibles con scroll horizontal.
RF-07	Modo oscuro	El usuario puede alternar entre tema claro y oscuro, persistiendo la preferencia en localStorage.
RF-08	Navegación responsive	La Navbar incluye menú hamburguesa con overlay completo en dispositivos móviles y tablets.

2.2 Requisitos No Funcionales (RNF)

ID	Requisito	Descripción
RNF-01	Accesibilidad WCAG AA	La interfaz cumple el nivel AA de las WCAG 2.1: contraste mínimo 4.5:1, navegación por teclado y atributos ARIA.
RNF-02	Responsive Design	La interfaz es funcional en móvil (320px), tablet (768px) y escritorio (1200px+).
RNF-03	Persistencia de sesión	Los datos de sesión y preferencias del usuario se mantienen entre recargas mediante localStorage.
RNF-04	Validación de formularios	Los formularios validan formato de email y complejidad de contraseña antes del envío.
RNF-05	Rendimiento	La aplicación carga en menos de 2 segundos gracias a Vite como bundler.
RNF-06	Modularidad	El código está organizado en componentes reutilizables con CSS Modules para evitar colisiones de estilos.

3. Planificación y Gestión del Tiempo

El proyecto se desarrolló en un plazo de 7 días naturales (del 11 al 17 de mayo de 2026), con una estimación de 6-8 horas efectivas de trabajo diario.

3.1 Desglose de Tareas (Backlog)

Fase	Tarea	Descripción	Estimación
Setup	Configuración del proyecto	Crear proyecto con Vite, instalar dependencias, configurar GitHub y estructura de carpetas.	3h
Componentes	Componentes base	Navbar, Footer, ItinerarioCard, Slider con flechas de navegación.	5h
Routing	React Router	Configurar rutas: Home, Itinerarios, Detalle, Login, Register.	2h
Estado	Hooks y Context	useState, useEffect, useLocalStorage, useDarkMode, AuthContext.	4h
Páginas	Páginas principales	Home con sliders, ItinerarioList, ItinerarioDetail, LoginForm, RegisterForm.	6h
Estilos	CSS Modules	Estilos responsive inspirados en Komoot, variables CSS, modo oscuro.	8h
Accesibilidad	WCAG AA	Atributos ARIA, focus visible, contraste, menú hamburguesa accesible.	3h
Validación	Formularios	Validación de email y contraseña con regex en Login y Register.	2h
Cierre	README y Git	Documentación del repositorio, merges a main, limpieza de ramas.	2h

4. Análisis Funcional y Flujos

4.1 Actores del Sistema

El sistema contempla dos tipos de usuario con acceso diferenciado:

Actor	Descripción	Acciones disponibles
Usuario no registrado	Visitante que accede sin cuenta.	Ver home, explorar itinerarios, ver detalle, registrarse, iniciar sesión.
Usuario registrado	Persona con cuenta activa y sesión iniciada.	Todo lo anterior + cerrar sesión, ver nombre en Navbar, alternar modo oscuro.

4.2 Interfaz de Usuario (UI)

La interfaz sigue una estructura consistente en todas las páginas:

- Navbar fija en la parte superior con logo, enlaces de navegación y acciones de sesión.
- Contenido principal centrado con un máximo de 1200px de ancho.
- Footer con información del proyecto en la parte inferior.

Las páginas principales son:

- Home: Hero con llamada a la acción + slider de itinerarios destacados + slider de ciudades.
- Itinerarios: Grid de 3 columnas (escritorio) / 2 columnas (tablet) / 1 columna (móvil).
- Detalle: Información completa del itinerario con metadatos en píldoras visuales.
- Login / Registro: Formulario centrado con validación en tiempo real.

5. Arquitectura y Stack Tecnológico

5.1 Arquitectura de Componentes

La aplicación sigue una arquitectura basada en componentes funcionales de React, organizada en capas:

Carpeta	Contenido	Responsabilidad
src/components/	Navbar, Footer, ItinerarioCard, Slider	Componentes reutilizables de UI.
src/pages/	Home, ItinerarioList, ItinerarioDetail, LoginForm, RegisterForm	Páginas completas asociadas a rutas.
src/hooks/	useLocalStorage, useDarkMode	Hooks personalizados con lógica reutilizable.
src/context/	AuthContext	Estado global de autenticación compartido entre componentes.
src/services/	itinerarios.js	Capa de abstracción para las llamadas a la API.
src/mocks/	itinerarios.js, ciudades.js, usuarios.js	Datos de prueba que simulan respuestas de la API.
public/images/	Imágenes de ciudades	Assets estáticos servidos directamente por Vite.

5.2 Stack Tecnológico y Justificación

Tecnología	Versión	Justificación
React	18+	Framework del módulo. Componentes funcionales con hooks para gestión de estado reactivo.
Vite	5+	Bundler moderno con HMR (Hot Module Replacement) que acelera el desarrollo frente a Create React App.
React Router v6	6+	Enrutamiento declarativo en cliente. Permite navegación sin recargas de página completa.
CSS Modules	-	Estilos con alcance local por componente. Evita colisiones sin necesidad de aprender Tailwind.
Context API	-	Estado global de sesión sin librerías externas. Suficiente para la escala del proyecto.
localStorage	-	Persistencia de sesión y preferencias entre recargas sin backend.
Git + GitHub	-	Control de versiones con flujo de ramas: main, dev, feature/*.

6. Accesibilidad y Usabilidad

6.1 Cumplimiento WCAG AA

Dado que SignaTour está diseñado para personas sordas o con discapacidad auditiva, la accesibilidad no es un añadido opcional sino un requisito central del proyecto. Se han implementado los siguientes criterios WCAG 2.1 nivel AA:

Criterio WCAG	Implementación
1.4.3 Contraste (mínimo)	Ratio mínimo 4.5:1 en todo el texto. Color primario #E8401C sobre blanco: ratio 4.6:1.
1.4.11 Contraste en componentes UI	Bordes de inputs y botones con contraste suficiente en ambos temas (claro y oscuro).
2.1.1 Teclado	Todos los elementos interactivos son accesibles por teclado. Focus visible con outline naranja de 3px.
2.4.3 Orden de foco	El orden de tabulación sigue el flujo visual lógico de la página.
2.4.6 Encabezados y etiquetas	Jerarquía de encabezados correcta (h1 > h2 > h3) en todas las páginas.
3.1.1 Idioma de la página	Atributo lang="es" declarado en el elemento html.
4.1.2 Nombre, función, valor	aria-label, aria-expanded, aria-controls y role declarados en elementos interactivos.
4.1.3 Mensajes de estado	role="alert" en mensajes de error de formularios para anuncio automático por lectores de pantalla.

Las pruebas de accesibilidad se realizaron con las herramientas:

- SiteImprove Accessibility Checker (extensión de navegador).
- Wave Evaluation Tool (extensión de navegador).
- Navegación manual por teclado en Chrome y Firefox.

6.2 Decisiones de Usabilidad

Las principales decisiones de diseño de experiencia de usuario tomadas durante el desarrollo:

- Patrón loading / error / data: todos los componentes que cargan datos muestran un estado de carga y uno de error antes de mostrar el contenido. Evita que el usuario vea una pantalla vacía.
- Sliders con scroll snap: el scroll horizontal de las tarjetas se detiene en cada tarjeta completa, evitando que el usuario se quede a medias entre dos cards.
- Flechas de navegación con estado desactivado: los botones de anterior/siguiente del slider se desactivan cuando no hay más contenido, dando retroalimentación clara al usuario.

- Menú hamburguesa con overlay completo: en móvil el menú ocupa toda la pantalla para facilitar el toque en zonas táctiles amplias (mínimo 44x44px por botón, recomendado por WCAG).
- Tarjetas completamente clicables: toda la superficie de la tarjeta es un enlace, no solo el botón interno, mejorando la usabilidad en dispositivos táctiles.
- Persistencia del modo oscuro: la preferencia de tema se guarda en localStorage para que el usuario no tenga que configurarlo en cada visita.
- Validación de formularios con mensajes claros: los errores indican exactamente qué campo falla y qué formato se espera.

7. Retos Técnicos y Soluciones

Reto 1: Sincronización del estado de sesión entre componentes

Situación: Al implementar el sistema de login, el estado del usuario se guardaba correctamente en localStorage, pero la Navbar no reflejaba el cambio después de iniciar sesión. El componente LoginForm guardaba el usuario, pero Navbar leía null porque cada componente tenía su propia instancia independiente del hook useLocalStorage.

Investigación: El problema radica en que React no propaga automáticamente los cambios de localStorage entre componentes. El evento storage del navegador solo se dispara entre pestañas distintas, no dentro de la misma pestaña.

Solución implementada: Se creó un AuthContext con React Context API que actúa como fuente única de verdad para el estado de sesión. Todos los componentes que necesitan saber si el usuario está logueado (Navbar, LoginForm, RegisterForm) consumen el contexto mediante un hook personalizado useAuth(), eliminando la dependencia de instancias individuales de localStorage.

Reto 2: Especificidad CSS en modo oscuro con CSS Modules

Situación: Al implementar el modo oscuro mediante el atributo data-theme="dark" en el elemento html, las reglas CSS definidas en index.css para sobrescribir colores de enlaces no surtían efecto. Las reglas aparecían tachadas en el inspector de Chrome.

Investigación: El problema era de especificidad CSS. La regla a { color: var(--color-primary) } definida globalmente tenía la misma especificidad que [data-theme="dark"] a { color: ... }, y CSS aplica la última regla declarada en caso de empate, no necesariamente la más específica para el contexto.

Solución implementada: Se aumentó la especificidad del selector de modo oscuro añadiendo body como ancestro: [data-theme="dark"] body a { color: #ee6b17 }. Para los componentes con CSS Modules, se utilizó el selector :global([data-theme="dark"]) .clase para acceder al contexto global desde un módulo local.

Reto 3: Accesibilidad del menú hamburguesa

Situación: El menú hamburguesa inicial no comunicaba correctamente su estado a los lectores de pantalla. Un usuario de tecnología asistiva no podía saber si el menú estaba abierto o cerrado, ni qué elemento controlaba qué.

Solución implementada: Se añadieron los atributos ARIA necesarios: aria-expanded indica si el menú está abierto (true/false), aria-controls conecta el botón con el id del menú que controla, aria-label cambia dinámicamente entre "Abrir menú" y "Cerrar menú" según el estado, y role="dialog" en el overlay identifica el menú como un diálogo modal para los lectores de pantalla.

8. Pruebas de Calidad (QA)

Se realizaron pruebas manuales de los flujos críticos de la aplicación, cubriendo el Happy Path (uso correcto) y los Edge Cases (casos límite y errores).

Caso de Prueba	Resultado Esperado	Estado
Login correcto	El sistema autentica al usuario y redirige a la home mostrando el nombre en la Navbar.	✓ Pasado
Login incorrecto	El sistema muestra "Email o contraseña incorrectos" sin revelar si el email existe.	✓ Pasado
Login con email inválido	El sistema muestra "Por favor, introduce un email válido" antes de consultar los datos.	✓ Pasado
Registro con contraseña débil	El sistema rechaza contraseñas sin mayúscula o sin número, con mensaje descriptivo.	✓ Pasado
Registro con contraseñas distintas	El sistema muestra "Las contraseñas no coinciden" sin crear la cuenta.	✓ Pasado
Persistencia de sesión	Al recargar la página, el usuario sigue logueado y la Navbar muestra su nombre.	✓ Pasado
Persistencia del modo oscuro	Al recargar la página, el tema oscuro se mantiene si estaba activado.	✓ Pasado
Navegación por teclado	Todos los elementos interactivos son accesibles con Tab y activables con Enter/Espacio.	✓ Pasado
Responsive móvil	En viewport de 375px el menú hamburguesa aparece y el grid pasa a 1 columna.	✓ Pasado
URL de itinerario inexistente	Al acceder a /itinerarios/999 se muestra el mensaje "Itinerario no encontrado".	✓ Pasado
Cierre de sesión	El botón "Cerrar sesión" elimina los datos de localStorage y redirige a /login.	✓ Pasado
Contraste en modo oscuro	Los textos sobre fondos oscuros mantienen un ratio de contraste mínimo de 4.5:1.	✓ Pasado

9. Conclusiones y Futuros Pasos

9.1 Conclusiones

El desarrollo de SignaTour Frontend ha permitido aplicar de forma práctica los conceptos fundamentales de React: componentes funcionales, hooks (useState, useEffect, hooks personalizados), Context API, React Router y CSS Modules. El proyecto cumple todos los requisitos básicos del enunciado y añade funcionalidades opcionales como el modo oscuro, la navegación por teclado y una interfaz accesible centrada en el usuario final.

La decisión de usar datos mock en lugar de conectar con el backend ha permitido centrarse en la calidad del frontend y en la accesibilidad, que es el valor diferencial del proyecto.

9.2 Mejoras Futuras

1. Conectar con el backend de SignaTour (Node.js + Express + PostgreSQL) para mostrar datos reales de la base de datos.
2. Implementar cambio de idioma (castellano, euskera, inglés) con la librería i18next.
3. Añadir una página de perfil de usuario donde gestionar sus datos e itinerarios favoritos.
4. Implementar un sistema de favoritos que permita guardar itinerarios de interés.
5. Añadir filtros en la lista de itinerarios por ciudad, duración y tipo de recurso de accesibilidad.
6. Integrar una API externa de puntos de interés culturales (OpenTripMap) para enriquecer el detalle de cada itinerario.
7. Desplegar la aplicación en Vercel o Netlify para acceso público.

10. Uso de la Inteligencia Artificial

10.1 Filosofía de Uso

Durante el desarrollo de SignaTour Frontend se ha utilizado Claude (Anthropic) como herramienta de apoyo al aprendizaje, no como generador automático del proyecto. La filosofía ha sido la de un estudiante que consulta a un profesor o mentor: se plantean preguntas, se reciben explicaciones y fragmentos de código, pero la comprensión, la integración y la toma de decisiones han sido propias.

10.2 Tareas para las que se ha usado la IA

- Resolución de errores específicos: cuando aparecía un error en la consola, se compartía el mensaje completo y se pedía una explicación de la causa y la solución.
- Explicación de conceptos nuevos: antes de implementar Context API o useRef, se pidió una explicación adaptada al nivel actual de conocimiento.
- Revisión de código: se compartían archivos completos para detectar problemas potenciales (por ejemplo, el problema de sincronización de localStorage entre componentes).

- Generación de datos mock: los datos de itinerarios y ciudades se generaron con ayuda de la IA para ahorrar tiempo en contenido que no es el foco del aprendizaje.
- Redacción de la memoria técnica: la estructura y el contenido base de este documento se han elaborado con apoyo de la IA, adaptando el contenido a la realidad del proyecto.

10.3 Tareas realizadas bajo criterio propio

- Todas las decisiones de arquitectura: qué componentes crear, cómo organizarlos, qué hooks usar en cada caso.
- Las decisiones de diseño visual: paleta de colores, tipografía, inspiración en Komoot, degradado verde en las cards.
- La integración del código: pegar un fragmento de código en el lugar correcto del archivo y entender cómo encaja con el resto.
- La detección de bugs: identificar que algo no funcionaba correctamente y describirlo con precisión para poder resolverlo.
- Las pruebas manuales: navegar por la aplicación, probar casos límite en los formularios y verificar el comportamiento responsive.

10.4 Aprendizaje del Uso Responsable de la IA

El uso de IA en el desarrollo de software plantea una responsabilidad clara: el código generado debe ser comprendido por quien lo integra. A lo largo del proyecto se ha pedido sistemáticamente una explicación de cada bloque de código antes de aplicarlo, lo que ha convertido la IA en una herramienta de aprendizaje acelerado en lugar de un atajo que omite la comprensión.

La IA comete errores (como ocurrió con comentarios JSX mal colocados dentro de paréntesis, o imports pegados en archivos CSS), y detectar esos errores requiere conocimiento propio. Esa capacidad crítica es la que distingue el uso responsable del uso irreflexivo de estas herramientas.

11. Aprendizajes Principales

El desarrollo de este proyecto ha consolidado los siguientes aprendizajes:

- React no es solo sintaxis: el mayor aprendizaje ha sido entender el modelo mental de React — los componentes son funciones que devuelven UI, el estado es inmutable, y los efectos secundarios tienen su lugar específico en `useEffect`.
- El flujo de datos en React es unidireccional: los datos bajan por props, los eventos suben por callbacks. Cuando varios componentes necesitan el mismo dato, la solución es Context API, no duplicar estado.
- La accesibilidad no es un añadido final: implementar `aria-label`, `role` y `focus visible` desde el principio es más fácil que añadirlo al final. En un proyecto sobre accesibilidad, este aprendizaje tiene doble valor.
- CSS Modules resuelve el problema de los estilos globales: nombrar clases como `.card` o `.titulo` en múltiples componentes sin que se interfieran entre sí es posible gracias a los módulos CSS.
- La especificidad CSS es un sistema de reglas, no magia: entender por qué una regla se aplica o no (y cómo arreglarlo) es una habilidad fundamental que este proyecto ha ayudado a desarrollar.
- Git con ramas `feature/` es una forma de trabajo profesional: trabajar con `feature/navbar`, `feature/login`, etc. y mergear a `dev` solo cuando algo funciona ha evitado romper código estable.